

Chapter 6

Scoring, Term Weighting & the Vector Space Model

Complete Discussion Notes

This document covers every topic from Chapter 6 in depth, including the Bag of Words model, Vector Space Model, Cosine Similarity, Term Frequency scaling, Document Frequency, IDF, TF-IDF weighting, and the SMART system with its notation scheme.

Topic 1 — Bag of Words (BoW)

What is Bag of Words?

Bag of Words is the simplest way to represent text in Information Retrieval. A document is treated as a collection of words where grammar, punctuation, and word order are completely ignored. Only which words appear and how many times they appear matter. Think of it like emptying a book into a bag — you lose the sequence, but you know exactly what words are inside.

A query is also transformed into the same BoW representation before comparison with documents.

Example

Given the following documents and query (after preprocessing — stopwords removal and stemming):

- D1 = {John, like, watch, movie, Mary, like, movie}
- D2 = {John, like, watch, football, game}
- D3 = {Mary, like, football, game}
- Q = {like, football, game}

Scoring Formula

Similarity is computed as the ratio of common terms to total unique terms (Jaccard-like measure):

$$\text{Score}(D, Q) = (D \cap Q) / (D \cup Q)$$

Document	Common Terms with Q	Union Terms	Score
D1	{like} — 1 term	8 unique terms	1/8 = 0.125
D2	{like, football, game} — 3 terms	5 unique terms	3/5 = 0.60
D3	{like, football, game} — 3 terms	4 unique terms	3/4 = 0.75

Notice why D1 scores so low: query terms 'football' and 'game' are absent, and the union is large because D1 has many unrelated terms. D3 wins because all 3 query terms match in a short document.

Preprocessing Note — Why 'to' and 'also' Disappear

Before building BoW, a standard preprocessing pipeline is applied: (1) lowercasing, (2) stopwords removal — common words like 'to', 'the', 'also', 'is' carry no meaning and are discarded, and (3) stemming — 'likes' becomes 'like', 'games' becomes 'game', 'movies' becomes 'movie'. This is why the raw sentence 'John also likes to watch football games' becomes simply {John, like, watch, football, game}.

Issues with BoW

Problem	Explanation
Document size ignored	A long document with 1 match scores worse than a short document with 1 match, even if content
Term frequency ignored	'football' appearing 10 times is treated the same as appearing once.
Terms treated as independent	'New York' is treated as 'New' + 'York' — semantic relationships between words are lost.

These three limitations motivated the development of the Vector Space Model.

Topic 2 — Vector Space Model (VSM)

The Big Idea

Instead of comparing word sets, VSM represents both documents and queries as mathematical vectors in a multi-dimensional space. Similarity is then a geometric property — how close two vectors point in the same direction. VSM was first introduced and tested in the SMART Information Retrieval System.

Building the Vocabulary Vector

From the entire document collection, build a dictionary of V unique terms (the vocabulary). Every document and query is then represented as a V -dimensional vector. Each dimension corresponds to one vocabulary term.

- Vocabulary = {football, game, John, like, Mary, movie, watch} — $V = 7$

	football	game	John	like	Mary	movie	watch
D1	0	0	1	1	1	1	1
D2	1	1	1	1	0	0	1
D3	1	1	0	1	1	0	0
Q	1	1	0	1	0	0	0

In this initial binary form, each cell is 1 if the term is present, 0 if absent. Later topics show how to replace these binary values with meaningful weights.

Key Assumptions of VSM

- Documents and queries are both represented as vectors in the same V -dimensional space.
- Relevant documents are geometrically close (small angle) to the query vector.
- All terms are statistically independent of each other.
- Term weighting can adjust how much each term contributes to similarity.

Why VSM is Better than BoW

Feature	Bag of Words	Vector Space Model
Similarity measure	Simple intersection ratio	Geometric cosine similarity
Term frequency	Ignored (binary)	Can be incorporated as weights
Document ranking	Crude ordering only	Continuous score, proper ranking

Feature	Bag of Words	Vector Space Model
Partial matching	Poor	Natural — via vector proximity

Topic 3 — Cosine Similarity

Why Not Euclidean Distance?

Euclidean distance measures the straight-line gap between two vectors. The problem: a long document that repeats the same terms as a short document will appear far away just because its vector values are numerically larger — even though both documents discuss the same topic. Document length skews the distance unfairly.

The Solution — Measure the Angle

Instead of measuring distance, VSM measures the angle (theta) between two vectors. Two vectors pointing in exactly the same direction have angle 0 and are perfectly similar. Two vectors pointing in perpendicular directions have angle 90 and share nothing in common. The cosine of this angle is used as the similarity score:

$$\text{Cos}(\text{theta}) = (\mathbf{d} \cdot \mathbf{q}) / (|\mathbf{d}| \times |\mathbf{q}|)$$

- $\mathbf{d} \cdot \mathbf{q}$ = dot product: multiply each matching dimension, then sum all products
- $|\mathbf{d}|$ = magnitude of document vector = $\sqrt{\text{sum of squares of all its weights}}$
- $|\mathbf{q}|$ = magnitude of query vector

Angle	Cosine Value	Meaning in IR
0 degrees	1.0	Identical vectors — perfect match
45 degrees	0.71	Moderate similarity
90 degrees	0.0	No common terms — completely different
180 degrees	-1.0	Opposite direction — rarely occurs in IR

Why Cosine Works Well

- Length-normalized: a long and short document about the same topic score equally because dividing by magnitudes cancels out length differences.
- Continuous score between 0 and 1 — easy to rank documents.
- Mathematically well-defined and computationally efficient.

Topic 4 — Term Frequency (TF)

The Problem with Binary Vectors

Binary vectors treat term presence as all-or-nothing: 1 if a term appears, 0 if not. But consider: Document A mentions 'football' once, Document B mentions 'football' ten times. Both get the same score of 1. Document B is clearly more focused on football, yet the model cannot distinguish them.

Term Frequency

TF simply counts how many times a term appears in a document. Instead of 1/0, the vector stores the actual count. This gives documents that discuss a topic more heavily a higher score for that term.

The Problem with Raw TF

Raw TF introduces a new problem: relevance does not scale linearly with frequency. A document where 'football' appears 100 times is not 100 times more relevant than one where it appears once. High-frequency terms would dominate the vector unfairly. The solution is sub-linear scaling.

Sub-linear TF Scaling Methods

Method	Formula	Example: tf=10	Notes
Binary	1 if tf>0, else 0	1	Ignores all frequency info
Raw TF	tf	10	Can dominate unfairly
Log Linear	$1 + \log(\text{tf})$	$1 + \log(10) = 2.0$	Most commonly used
Augmented	$0.5 + 0.5 \times \text{tf} / \text{max_tf}$	Depends on max_tf	Stays between 0.5 and 1.0
Max Freq Norm	tf / max_tf in doc	Relative to dominant term	Normalizes within doc

Log linear scaling is the most popular because it grows slowly and predictably: tf=1 gives 1.0, tf=10 gives 2.0, tf=100 gives 3.0 — relevance increases but never explodes.

Augmented TF is useful for longer documents where you want to ensure no single term dominates. By dividing by the maximum frequency in the document, everything is normalized to the [0.5, 1.0] range.

Topic 5 — Document Frequency & IDF

The Problem TF Alone Cannot Solve

Even with perfect TF scaling, a fundamental problem remains. Some words appear in almost every document — words like 'computer' in a CS collection, or 'said' in a news collection. These words have high TF in many documents, but they provide no power to distinguish relevant from irrelevant results.

Document Frequency (DF)

DF counts the number of documents in the entire collection that contain a given term. A high DF means a term is common and therefore less useful for discrimination. A low DF means a term is rare and highly discriminating.

Term	DF (out of 10,000 docs)	Usefulness
the	~10,000	Useless — appears everywhere
computer	~5,000	Low — too common in CS corpus
football	~200	Moderate — useful discriminator
neuromorphic	~3	Excellent — very rare and specific

Inverse Document Frequency (IDF)

IDF flips document frequency: rare terms get high weight, common terms get low weight. The formula uses a log to keep values reasonable:

$$\text{idf}(t) = \log(N / \text{df}(t))$$

Where N = total documents in the collection, df(t) = number of documents containing term t.

Term	DF	IDF = $\log(10000/\text{df})$	Effect
the	10,000	$\log(1) = 0$	Completely eliminated
computer	1,000	$\log(10) = 1.0$	Low weight

Term	DF	IDF = $\log(10000/df)$	Effect
football	100	$\log(100) = 2.0$	Moderate weight
neuromorphic	10	$\log(1000) = 3.0$	High weight

Notice how 'the' gets completely eliminated with IDF = 0. This is exactly the desired behavior — stopwords are mathematically neutralized without needing an explicit stopwords list.

Augmented IDF

$$\text{Augmented IDF} = \max\{ 0, \log((N - df) / df) \}$$

The augmented version adds a safety floor of 0, preventing negative IDF values which can occur when $df > N/2$. This handles edge cases gracefully.

Topic 6 — TF-IDF Weighting

The Grand Combination

TF and IDF are combined multiplicatively to produce a single powerful weight for each term in each document:

$$\text{tf-idf}(t, d) = \text{tf}(t, d) \times \text{idf}(t)$$

This formula elegantly balances two complementary signals: TF rewards terms that appear frequently within a specific document, while IDF penalizes terms that appear across many documents. Together they identify terms that are both locally frequent and globally rare — the most discriminating terms possible.

The Three Golden Rules of TF-IDF

Situation	TF-IDF Score	Reason
Term appears many times in few documents	HIGHEST	Highly discriminating and strongly relevant
Term appears few times, or in many documents	MEDIUM	Weaker relevance signal
Term appears in virtually all documents	LOWEST (~0)	No discriminating power at all

Concrete Example (N = 10,000 documents)

Query: 'football neuromorphic' — Document D1 contains 'football' 5 times and 'neuromorphic' 3 times.

Term	TF in D1	DF	IDF = $\log(N/df)$	TF-IDF
football	5	5,000	$\log(2) = 0.30$	$5 \times 0.30 = 1.50$
neuromorphic	3	10	$\log(1000) = 3.0$	$3 \times 3.0 = 9.00$

Even though 'football' appeared more times in the document, 'neuromorphic' carries 6x more weight because it is so rare in the collection. This is the power of TF-IDF — it rewards specificity.

Different Variants — SMART Table

The SMART system catalogued all possible TF, IDF, and normalization combinations using a standard notation. The three components are:

Component	Letter	Name	Formula
Term Freq	n	Natural	$\text{tf}(t,d)$
Term Freq	l	Logarithm	$1 + \log(\text{tf})$
Term Freq	a	Augmented	$0.5 + 0.5 \times \text{tf} / \text{max_tf}$

Component	Letter	Name	Formula
Term Freq	b	Boolean	1 if $tf > 0$, else 0
Doc Freq	n	None	1
Doc Freq	t	IDF	$\log(N/df)$
Doc Freq	p	Prob IDF	$\max\{0, \log((N-df)/df)\}$
Norm	n	None	1
Norm	c	Cosine	$1 / \sqrt{w_1^2 + w_2^2 + \dots + w_M^2}$
Norm	u	Pivoted	$1/u$ (pivoted unique)

Topic 7 — Advantages & Disadvantages of VSM

Advantages

Advantage	Explanation
Simple linear algebra	Built entirely on vectors and dot products. Easy to implement and computationally efficient.
Non-binary term weights	TF-IDF replaces 0/1 with meaningful continuous weights reflecting true term importance.
Continuous similarity score	Cosine gives a score between 0 and 1 — not just relevant/not-relevant like the Boolean model.
Document ranking	Documents can be sorted by score so the most relevant appear first — the basis of modern search.
Partial matching	A document matching only some query terms still receives a meaningful score and can be retrieved.
Proven track record	Over 20 years of research and real-world deployment validate the model's effectiveness.

Disadvantages

Disadvantage	Explanation
Word order is lost	'Dog bites man' and 'Man bites dog' produce identical vectors. Sequence and grammar are invisible.
Terms assumed independent	VSM has no concept of 'New York' as a single entity, or that 'ice' and 'cold' are related.
Exact term matching required	Search keywords must precisely match document terms. Substrings can cause false positive matches.
Semantic insensitivity	Documents about 'automobiles' and queries about 'cars' score near zero despite meaning the same.

These semantic limitations of VSM are what motivated modern techniques like Latent Semantic Analysis (LSA), Word2Vec, and BERT-based retrieval models.

VSM vs Boolean Model — Final Comparison

Feature	Boolean Model	Vector Space Model
Term weights	Binary (0 or 1)	Continuous (TF-IDF)
Similarity measure	Exact match only	Cosine similarity (0 to 1)
Document ranking	Not possible	Natural — sort by score
Partial matching	Not supported	Fully supported
Word order preserved	No	No
Semantic understanding	No	No

Topic 8 — The SMART System

What is SMART?

SMART (System for the Mechanical Analysis and Retrieval of Text) was one of the first major IR research systems, developed by Gerard Salton at Cornell University in the 1960s. It was the birthplace of the Vector Space Model and TF-IDF. SMART rigorously experimented with different combinations of weighting schemes to determine which work best in practice, producing a standardized notation system still used today.

The ddd.qqq Notation

Every weighting scheme is described as six letters in two triplets: ddd.qqq. The first triplet describes how the document vector is weighted; the second describes how the query vector is weighted. Within each triplet, the three letters specify: (1) the TF component, (2) the IDF component, and (3) the normalization method.

Example: l n c . l t c

Position	Document (Inc)	Query (Itc)
TF component	l — log weighted TF	l — log weighted TF
IDF component	n — no IDF applied	t — standard IDF weighting
Normalization	c — cosine normalization	c — cosine normalization

Why Inc.Itc is the Standard Scheme

For documents (Inc): Log TF prevents high-frequency terms from dominating. No IDF is applied because documents are pre-processed offline and IDF at query time is sufficient — applying it twice is redundant and wasteful. Cosine normalization accounts for varying document lengths.

For queries (Itc): Log TF keeps query term counts sub-linear. IDF is applied here to boost rare query terms and penalize common ones. Cosine normalization keeps the query vector comparable in scale to document vectors.

Step-by-Step Inc.Itc Scoring

Given N=3 documents and Query = 'like football':

Step	Action	Details
1	Compute document weights (Inc)	For each term in doc: weight = (1+log(tf)) x 1, then cosine-normalize the whole vector
2	Compute query weights (Itc)	For each query term: weight = (1+log(tf)) x log(N/df), then cosine-normalize
3	Compute cosine similarity	Sum over matching terms: doc_weight x query_weight
4	Rank documents	Sort all documents by their cosine similarity score descending

Why SMART Still Matters Today

SMART Contribution	Modern Equivalent
TF-IDF weighting scheme	Core of Elasticsearch, Apache Lucene, and Solr
Cosine similarity measure	Foundation of recommendation systems and neural IR
Systematic IR experimentation	Basis of TREC (Text REtrieval Conference) evaluations
Vector Space Model framework	Conceptual ancestor of Word2Vec, GloVe, and BERT

Chapter 6 — Complete Summary

Topic	Core Concept	Key Formula / Idea
Bag of Words	Represent text as word counts, discard order	$\text{Score} = D \cap Q / D \cup Q $
Vector Space Model	Docs and queries as V-dimensional vectors	$d = \langle w_1, w_2, \dots, w_V \rangle$
Cosine Similarity	Measure angle between vectors, not distance	$\text{Cos}(\theta) = d \cdot q / (d q)$
Term Frequency	Count occurrences; scale sub-linearly	$1 + \log(\text{tf})$
Document Frequency	Count docs containing a term	$\text{df}(t)$
IDF	Penalize common terms, reward rare ones	$\text{idf} = \log(N / \text{df})$
TF-IDF	Combine local frequency with global rarity	$\text{tf-idf} = \text{tf} \times \text{idf}$
SMART System	Standardized notation for weighting schemes	Inc.ltc is the gold standard

These eight topics form the mathematical foundation of modern Information Retrieval. The limitations of each model — BoW's ignorance of frequency, VSM's assumption of term independence, TF-IDF's inability to capture semantics — directly motivated the next generation of retrieval models including Latent Semantic Analysis, probabilistic models, and today's transformer-based neural retrieval systems.